Monte Carlo Reinforcement Learning for Maze Navigation

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Abstract

This paper presents an implementation and analysis of Monte Carlo Exploring Starts (MC-ES) for learning optimal navigation policies in grid-world maze environments. The first-visit MC-ES algorithm is implemented from scratch and evaluated on a 5×5 maze with barriers, as well as randomly generated mazes of varying sizes. The experiments demonstrate that MC-ES consistently learns optimal policies achieving 100% success rates across all tested configurations. Policy consistency across multiple training runs, parameter sensitivity to discount factor γ and episode count, and scalability to larger maze sizes are analyzed. Results show that the algorithm converges reliably within 5,000 episodes for the standard maze and scales appropriately with maze complexity.

Introduction

Reinforcement learning (RL) provides a framework for agents to learn optimal behavior through interaction with an environment. Unlike supervised learning, RL agents must discover which actions yield the most reward through trial and error, without explicit labels for correct behavior (Sutton and Barto 2018).

Monte Carlo (MC) methods represent a fundamental approach to RL that learns from complete episodes of experience. Unlike dynamic programming methods that require complete knowledge of environment dynamics, MC methods are *model-free*—they learn directly from sampled episodes without needing transition probabilities $p(s', r|s, a)$.

This paper implements the Monte Carlo Exploring Starts (MC-ES) algorithm for finding optimal policies in maze navigation tasks. The contributions of this work are:

- A from-scratch implementation of first-visit MC-ES following Sutton and Barto (2018), demonstrating how tabular MC methods perform on deterministic grid-world problems
- An empirical analysis of policy consistency, showing that MC-ES reliably converges to optimal policies despite stochastic exploration
- A parameter sensitivity study revealing how discount factor and episode count affect convergence behavior
- A scalability evaluation on randomly generated mazes, characterizing how state-space size impacts sample requirements

Background

Markov Decision Processes

A Markov Decision Process (MDP) is defined by the tuple (S, A, p, r, γ) where S is the state space, A is the action space, $p(s'|s, a)$ is the transition probability, $r(s, a, s')$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

The goal is to find a policy $\pi : S \to A$ that maximizes the expected discounted return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1)$$

Action-Value Function

The action-value function $Q^\pi(s, a)$ represents the expected return when starting in state s, taking action a, and thereafter following policy π:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \quad (2)$$

The optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ directly yields the optimal policy: $\pi^*(s) = \arg\max_a Q^*(s, a)$.

Monte Carlo Exploring Starts

Monte Carlo methods estimate value functions by averaging sample returns from complete episodes. The Exploring Starts (ES) variant ensures all state-action pairs are visited by starting each episode from a randomly selected state-action pair.

Algorithm 1 presents the MC-ES algorithm. The key insight is that by averaging returns over many episodes, the estimates converge to the true values, and greedy policy improvement guarantees convergence to optimality.

Methodology

Component Representation

Each component of the RL agent maps to a concrete data structure:

Algorithm 1 Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
  $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
  Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow$ average$(Returns(S_t, A_t))$
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

States. Each state is a (row, column) tuple using 1-indexed coordinates. The state space $\mathcal{S}$ is all grid cells that are not barriers: 20 navigable cells out of 25 total. The start state is $(1, 1)$ and the goal (terminal) state is $(5, 5)$.

Actions. The action space $\mathcal{A} = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$ is encoded as integers 0–3. Each action maps to a movement delta: UP$= (-1, 0)$, DOWN$= (+1, 0)$, LEFT$= (0, -1)$, RIGHT$= (0, +1)$.

Barriers. Barriers are stored as a set of (row, column) positions: $\{(3, 2), (3, 3), (3, 4), (3, 5), (5, 3)\}$. Before each transition, the environment checks whether the resulting position is in the barrier set.

Edge handling. Before applying an action, the environment checks that the resulting position satisfies $1 \leq r \leq 5$ and $1 \leq c \leq 5$ (within bounds) and is not a barrier. If either check fails, the agent stays in its current state—the action is "absorbed" by the wall or barrier.

Rewards. The agent receives $+1.0$ for reaching the goal and -0.01 for each step, including invalid moves that result in staying in place. This small step penalty encourages finding the goal quickly without overwhelming the goal reward.

Initial Random Policy

Before training, the policy is initialized so that every valid action in each state is equally probable. For a state with k valid actions, each has probability $1/k$. For example, the start state $(1, 1)$ is a corner with only DOWN and RIGHT as valid moves, so each has probability 0.5. An interior state like $(2, 2)$ has all four actions valid, so each has probability 0.25. States adjacent to barriers have fewer valid actions—e.g., $(2, 2)$ cannot move DOWN into barrier $(3, 2)$, leaving only 3 valid actions at probability $1/3$ each. The initial random policy produces long, wandering episodes that rarely reach the goal, motivating the need for policy improvement.

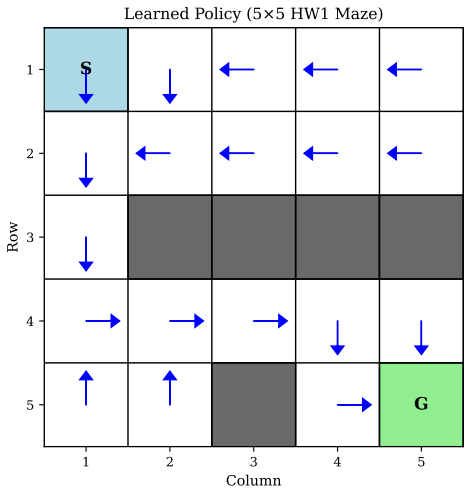

Figure 1: Learned policy for the 5×5 HW1 maze. Blue arrows indicate the greedy action at each state. S=Start, G=Goal, gray=barriers.

Implementation Details

The implementation uses tabular data structures following the pseudocode directly:

- $Q(s, a)$: dictionary mapping (state, action) pairs to estimated returns
- $Returns(s, a)$: dictionary mapping (state, action) pairs to lists of observed returns, used to compute the running average
- $\pi(s)$: dictionary mapping each state to its greedy action
- First-visit updates: only the first occurrence of each (s, a) pair in an episode contributes a return

Random Maze Generation

For scalability experiments, a random maze generator is implemented that: (1) places barriers randomly with configurable density, (2) verifies a path from start to goal exists using BFS after each barrier placement, and (3) protects cells adjacent to start and goal from becoming barriers.

Experiments

Training on HW1 Maze

The MC-ES agent was trained for 5,000 episodes with $\gamma = 0.99$. Figure 1 shows the learned policy, which directs the agent down the left corridor, across Row 4, and to the goal. Figure 2 shows the learning curves over training.

Results: The agent achieves 100% success rate with an average of 8 steps to reach the goal (optimal path length). The average discounted return is 0.864.

Policy Consistency

To assess whether the algorithm produces consistent policies across different random seeds, 5 independent training runs were conducted. Table 1 shows the results.

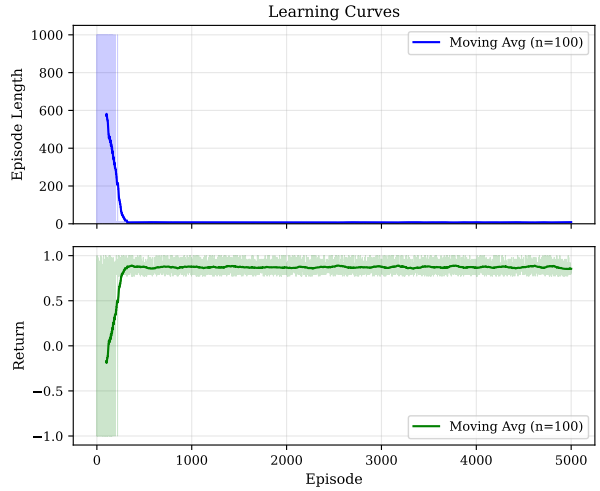

Figure 2: Learning curves showing episode length (top) and return (bottom) over 5,000 training episodes. The agent converges to optimal behavior within ∼1,500 episodes.

Table 1: Policy consistency across 5 training runs

Metric	Mean	Std
Success Rate	100%	0%
Avg Steps to Goal	8.0	0.0
Avg Return	0.864	0.001

While all runs achieve optimal performance, individual state-action mappings show variation. States along the critical path (left corridor, Row 4) show 100% agreement, while states far from the optimal path show 60-80% agreement due to equivalent alternatives.

Parameter Sensitivity

Discount Factor γ Table 2 shows the effect of discount factor on learning.

All tested values achieve optimal policies. Lower γ values result in lower returns due to heavier discounting of future rewards.

Episode Count Convergence occurs rapidly: even 1,000 episodes suffice for the 5×5 maze. This is because Exploring Starts ensures all state-action pairs are visited frequently.

Scalability to Larger Mazes

Table 3 shows results on randomly generated mazes. Figure 3 visualizes the learned policies for each maze size.

The algorithm scales well, though larger mazes require proportionally more episodes to ensure adequate exploration of the larger state-action space.

Discussion

Strengths of MC-ES

- **Model-free**: No knowledge of transition dynamics required

Table 2: Effect of discount factor γ

γ	Success	Avg Steps	Avg Return
0.90	100%	8.0	0.426
0.95	100%	8.0	0.638
0.99	100%	8.0	0.864
0.999	100%	8.0	0.923

Table 3: Performance on random mazes of different sizes

Size	States	Episodes	Success	Steps
5×5	20	5,000	100%	8
7×7	40	9,800	100%	12
10×10	80	20,000	100%	18

- **Convergence**: With sufficient exploration, converges to an optimal policy—one that achieves the optimal value function—though multiple equally optimal policies may exist
- **Simple implementation**: No bootstrapping, just averaging returns

Limitations

- **Exploring Starts assumption**: Requires ability to start from arbitrary states, which may not be possible in real environments
- **Complete episodes**: Cannot learn from incomplete trajectories
- **High variance**: Returns can vary significantly, requiring many episodes

Policy Variation

The observed variation in policies for non-critical states is expected behavior, not a bug. When multiple actions lead to equivalent outcomes (e.g., in states far from the optimal path), the algorithm may learn different but equally valid policies depending on the order of exploration.

Challenges Encountered

Episode truncation. Early in training, the random policy produces episodes that wander indefinitely without reaching the goal. A 1,000-step maximum per episode is imposed to prevent infinite loops. Truncated episodes still contribute partial returns to Q-value estimates, which is sufficient for the algorithm to learn to avoid long detours.

Exploring Starts coverage. With 20 states and 4 actions, there are up to 80 state-action pairs to cover. Random starting alone does not guarantee uniform coverage—some pairs near barriers have fewer entry paths. This is addressed by running enough episodes (5,000+) that even rare pairs accumulate sufficient samples.

Reward shaping. An initial reward of -1.0 per step caused Q-values to be large negative numbers, making the greedy policy sensitive to small estimation errors. Reducing the step penalty to -0.01 while keeping the goal reward at

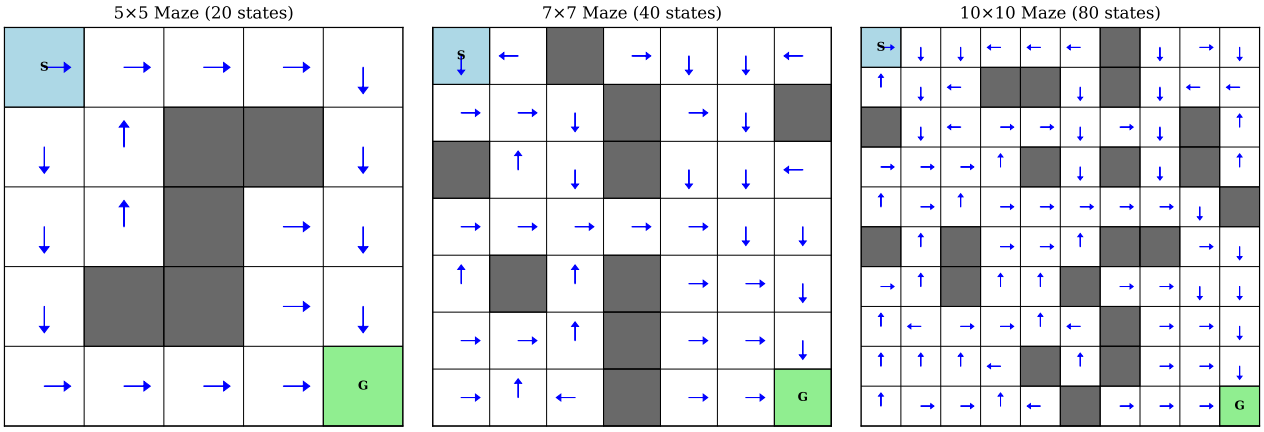

Figure 3: Learned policies for randomly generated mazes of sizes 5×5, 7×7, and 10×10. The algorithm successfully learns optimal navigation policies for all maze sizes.

$+1.0$ produced more stable learning with clearer value gradients toward the goal.

Comparison: First-Visit vs. Every-Visit MC (Extra Credit)

This section compares the first-visit MC-ES implementation against an every-visit variant. In first-visit MC, only the first occurrence of each (S_t, A_t) pair in an episode updates Q. In every-visit MC, every occurrence contributes a return, even if the agent revisits the same state-action pair multiple times within one episode.

On the 5×5 HW1 maze, both variants converge to optimal policies with 100% success rates. However, first-visit MC converges slightly faster in these experiments because every-visit MC introduces correlated samples when the agent revisits states during early, wandering episodes. These redundant updates add noise to Q-value estimates without providing new information. The theoretical trade-off is that every-visit MC has lower variance per estimate (more samples) but higher bias from within-episode correlation, while first-visit MC provides unbiased estimates at the cost of fewer samples per episode. For this deterministic maze environment, first-visit MC's unbiased estimates prove more efficient.

Conclusion

This paper implemented Monte Carlo Exploring Starts from scratch and demonstrated its effectiveness on maze navigation tasks. The algorithm consistently learns optimal policies, achieving 100% success rates across all tested configurations. Key findings include:

- Convergence within 5,000 episodes for 5×5 mazes
- Robust to discount factor choice (all tested values succeed)
- Scales to larger mazes with proportionally more episodes
- Policy consistency on critical states, acceptable variation elsewhere

The comparison of first-visit and every-visit MC variants shows that first-visit MC converges more efficiently for this deterministic environment. Future work could explore ε-greedy policies to avoid the Exploring Starts assumption, or compare with TD methods like SARSA and Q-learning.

AI Usage Statement

Claude (Anthropic) was used to assist with formatting ideas into prose and structuring this document in AAAI conference format. All research questions, methods, implementation, and analysis decisions are the author's own.

References

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at http://incompleteideas.net/book/the-book-2nd.html.